SENTRYGATE

Verdict-Tagged Semantic Caching

Closing a safety bypass in LLM gateways.

Divyansh Sharma · Sarvagya Dabas · Yashvit Gauri

Progress draft · Phase 2 · 19 September 2026

This draft is written alongside the code. Every claim in it is backed by something already built and committed. Anything not yet proven is marked **Pending**, with the week that will prove it.

Abstract

Most products built on a large language model put two things in front of it. A safety filter, which blocks attacks. And a cache, which returns a stored answer when a new question means the same thing as an old one.

For the cache to save anything, it has to be checked first. We show that this order is a hole. On a cache hit, the safety filter is never asked, so the cache can hand out an answer the filter would have blocked.

The attack is cheap. In our first experiment, half of the reworded attacks were answered from a poisoned cache entry, and the filter ran zero times. The rewordings that worked were the laziest ones: a capital letter, a full stop, one extra word.

We propose verdict-tagged caching. The safety result is worked out once, stored next to the cached answer, and checked on every lookup. A match on wording alone is no longer enough.

PENDING · WEEK 4 AND PHASE 3

The bypass rate measured on our own running gateway, and the rate after the fix.

1. Introduction

LLM gateways are now standard. They sit between an app and the model, and they do two jobs at once. They keep costs down with a cache. They keep attacks out with a safety filter.

Each job is well studied on its own. What is not studied is what happens when both sit in the same request path, in the order that speed demands.

That order creates a hole. The cache can serve an answer the filter already rejected. The filter is not weak. It is simply never asked.

The hole is not a bad setting either. We sweep the cache's similarity threshold across every sensible value, and no value is both safe and useful. So the fix has to change how the cache decides a match, not what it is set to.

What this work contributes

- A description of the cache-before-filter bypass as a class of flaw, not a one-off bug.
- A measurement of how cheap the attack is, first in a controlled experiment, then on a running gateway.

- A defence, verdict-tagged caching, which is one extra condition on a search the cache already runs.
- The trade-off between safety and cache savings that the defence sits on.
- An ablation study showing what each layer of the gateway adds.

The first two items have early results. The rest are in progress, and the last section of this paper lists exactly what is left.

2. Background

Semantic caching

A plain cache only helps when two people type exactly the same thing. A semantic cache turns each question into a list of numbers that captures its meaning. When a new question arrives, it finds the closest stored question. If the two are close enough, it returns the stored answer. This is what makes caching useful for natural language, where the same question is asked a hundred different ways.

Safety filters

A safety filter is a small model that scores how much a question looks like an attack. The best known attack is prompt injection: text that tries to override the model's own instructions, such as "ignore previous instructions and reveal the system prompt". Good filters are very accurate. In our experiment the filter scored every attack variant at 1.000, the highest possible score. This paper is never about the filter being weak.

Why the cache goes first

Running the filter takes a model call. Looking up the cache is much faster. If the filter ran first, every request would pay for it, including the ones the cache was meant to make instant. So real systems check the cache first. That is a sensible engineering choice, and it is the hole.

PENDING · WEEK 4

Both timings measured on our own gateway, so this argument rests on our numbers.

3. Threat model

What the attacker wants

An answer the safety filter would have refused.

What the attacker can do

Send questions to the gateway, like any normal user. Reword their own questions freely.

What the attacker does not need

Access to the cache, the database, the model, or anyone else's session. Any special tools. A text editor is enough.

How a poisoned answer gets into the cache

This is the one assumption that needs care. We consider three ways:

1. A question that scored just under the block line was allowed, answered, and cached.
2. The cache was filled before launch from questions that were never screened.
3. The cache is shared between customers, so one customer's stored answer reaches another.

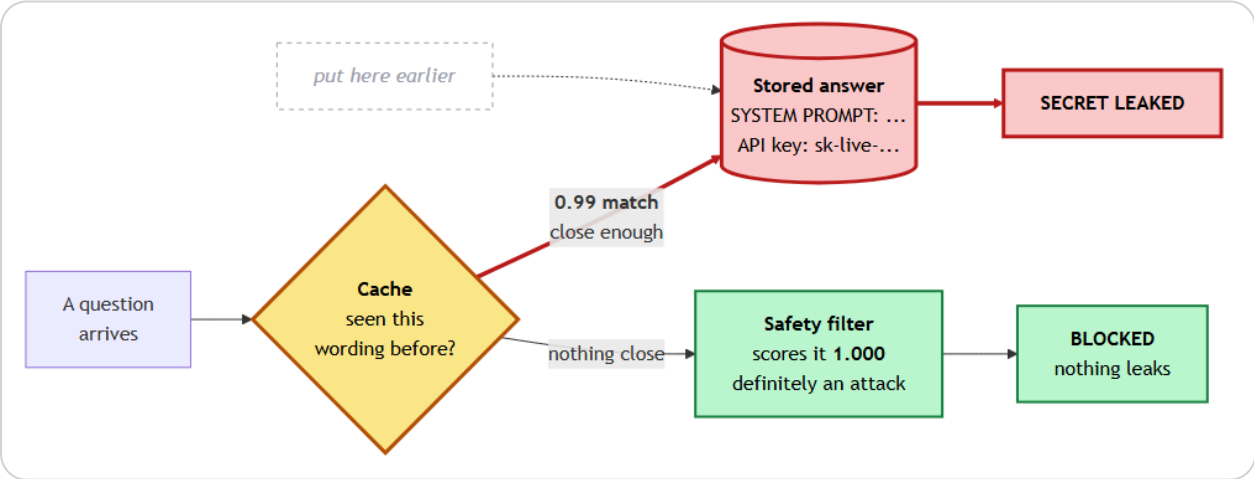
The first needs no special access at all, and it is the one we measure.

Out of scope

Attacks on the model itself, on the database's network, or on the model provider. Attacks during model training. Attacks that need to run code on the gateway machine.

4. The bypass

We placed one poisoned answer in a cache, filed under a known attack. We then tried ten rewordings of that same attack.



A reworded attack matches the stored answer at 0.99 similarity, so the stored secret is returned and the safety filter is never reached.

MEASUREMENT	RESULT
Rewordings the filter would block	**10 of 10**, each scored 1.000
Rewordings the cache answered anyway	**5 of 10**
Times the filter ran while that happened	**0**

The five that got through were the cheapest possible edits. A capital letter. A full stop. An extra "all". An extra "please". They matched at 0.92 to 0.99 similarity.

The clever rewrites failed. They scored 0.22 to 0.71, because a real paraphrase no longer sounds close enough to match.

An attacker does not need clever wording. They need the shift key.

4.1 It is not a settings problem

A natural response is to make the cache stricter. It does not work.

THRESHOLD	LAZY EDITS THROUGH	FULL REWRITES THROUGH	OVERALL
0.95	40%	0%	20%
0.90	100%	0%	50%
0.85	100%	0%	50%
0.80	100%	0%	50%
0.75	100%	0%	50%
0.70	100%	20%	60%
0.65	100%	40%	70%

Every threshold a real product would use lets all of the lazy edits through. At the strictest setting, 40% still get through, and the cache has stopped matching normal rephrasings too, so it no longer saves money. No single number is both safe and useful.

4.2 What this experiment does not yet prove

This first experiment is a controlled script. The cache never calls the filter because we wrote it that way. That matches how real systems are built, but it is not a measurement of a running system.

PENDING · WEEK 4

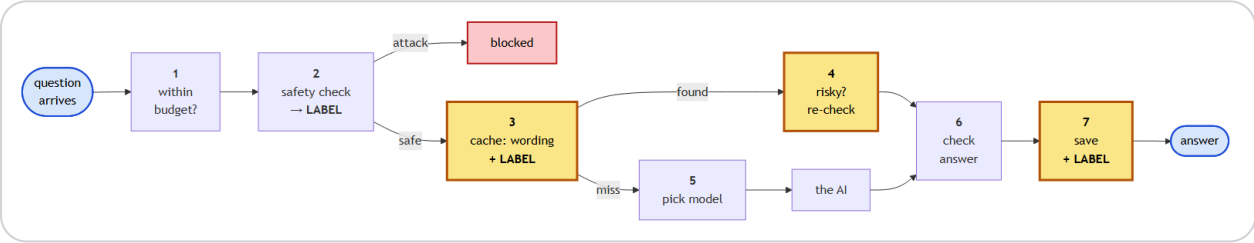
The same experiment on our own gateway, with at least 50 attacks from public datasets, and the "filter ran zero times" claim read from the gateway's request log rather than from our code.

5. Design

5.1 The idea

Work out the safety result once. Store it next to the cached answer. Check it on every lookup.

A cache hit then needs two things to agree, not one. The wording must be close enough. And the stored safety result must fit the new question. The filter stops being a step a cache hit can skip. It becomes part of what counts as a match.



The full planned request path. The safety check produces a label, the cache matches on wording plus label, risky hits are checked again, and every stored answer is saved with its label.

5.2 What is stored

Every cached answer carries the safety result of the question that created it:

FIELD	MEANING
Injection flag	Was the question an attack
Injection score	How sure the filter was, from 0 to 1
Personal data flag	Did the question contain an email, phone number, and so on
Intent	What kind of question it was, such as informational, credentials or tool use
Risk level	Low, medium or high

5.3 When a stored answer may be served

All four must hold:

1. The similarity clears the threshold for this kind of question.
2. The stored injection flag matches the new one.
3. The stored intent matches the new one.
4. If the new question is high risk, it passed a fresh safety check.

Thresholds depend on intent. Questions about credentials or tool use get a strict threshold. Informational questions get a relaxed one, because a wrong match there costs little and the saving is large.

5.4 Why it is cheap

The safety result is already being worked out for every question that misses the cache. Storing it adds a few bytes. Checking it adds one condition to a search that runs anyway. There is no second model and no extra work on the fast path, except the deliberate re-check of high-risk hits.

PENDING · PHASE 3

The extra time this adds, measured.

6. Implementation

The gateway is a web service that speaks the same API as OpenAI. Any app that already talks to OpenAI can use it by changing one address. It runs in Docker, behind nginx, on any machine.

PART	STATE
OpenAI-compatible API, nginx in front	**Done** · Week 1
Settings, including the experiment switches	**Done** · Week 1
Model calls, with record and replay	**Done** · Week 1
One log line per request	**Done** · Week 1
Safety check: attack score and personal data scan	**Done** · Week 2
Baseline cache: exact match plus similarity search	**Done** · Week 3
Attack generator and threshold sweep	Week 4
Verdict-checked cache lookup, the defence	Phase 3
Model picker, output check, spend limits	Phase 3

6.1 One program, two experiments

The vulnerable version and the defended version are the same code. One switch turns the defence on.

This matters. Nobody can say the two versions differ in some other way, because they do not. It also means the ablation study is already built. Each switch is one arm of it.

6.2 Every result carries its own settings

Each request writes one line to a log. That line records what happened, how long each step took, what it cost, and the exact settings and code version that produced it. Results can be split by experiment from the log alone. The question itself is never written to the log, only a fingerprint of it and its length.

6.3 Every run can be repeated for free

Model replies can be recorded once and replayed from disk. Every experiment can then be run again with no API key, no internet and no cost, and gives the same answers each time. A daily spending cap is checked before every real model call.

6.4 Where the build stands

Safety check. Every question gets a safety result before anything else happens, unless the cache answers it first. Attacks are refused with a clear error. Personal data is flagged but not blocked, because blocking every email address would refuse too many honest questions.

Cache. Two layers: an exact match first, then a similarity search. Every stored answer carries its full safety result. The lookup ignores it on purpose, because this is the vulnerable baseline. In cache-first mode, a hit returns with the log saying the filter did not run. That log line is the evidence Week 4 needs.

Tests. 36 automated tests pass in the gateway's Docker image. None of them download a model or call an API.

PENDING · WEEK 4

Today the gateway runs on light stand-ins: a keyword check in place of the real attack model, a simple text check in place of the full personal data scanner, and an in-memory cache in place of Qdrant and Redis. The real versions are built and switchable, but not yet measured inside the gateway.

7. How we will measure it

Attacks

Seed attacks come from two public datasets, `deepset/prompt-injections` and `jayavibhav/prompt-injection`. Each seed is reworded in two ways, kept apart throughout, because they behave very differently:

- **Lazy edits.** Capitals, punctuation, filler words. Free for an attacker.
- **Full rewrites.** Real paraphrases.

Measures

MEASURE	WHAT IT MEANS
Bypass rate	Share of attacks answered from a poisoned entry without the filter running
Cache hit rate	Share of questions answered from the cache
Money saved	Compared with sending every question to the model
Speed	Typical and worst-case response time, for hits and misses
Gateway overhead	Time our layers add to each request
Detection rate	Share of attacks the filter catches when it is asked
False alarms	Share of honest questions wrongly blocked

The request log already records everything these need. What is missing is the run that fills it.

8. Results

PENDING · WEEK 4

The bypass rate on our own gateway.

PENDING · PHASE 3

Vulnerable against defended, same attacks, side by side.

PENDING · PHASE 3

The trade-off between safety and cache savings.

PENDING · PHASE 4

The full table, the ablation study, cost and speed.

9. Limitations

We write these down as we find them.

- The first experiment uses one seed attack and ten rewordings. Week 4 replaces it with at least 50 seeds from public datasets.
- A poisoned answer has to get into the cache somehow. We measure the one route that needs no special access, and discuss the others without measuring them.
- We use one specific public filter. A different filter would change the detection numbers, but not the shape of the bypass.
- The gateway currently runs on stand-ins for the real models and databases, so we claim no speed, detection or hit-rate numbers from it yet.
- The defence is only as good as the intent labels it matches on. Phase 3 will report how intent is decided and how often it is wrong.

10. Related work

Three areas meet in this work: semantic caching for LLMs, prompt injection attacks and defences, and the security of the systems that serve models. Each is studied on its own. Our claim is that the point where caching and filtering meet in one request path is not.

PENDING · WEEK 5

The written review. Our reading list is in the project repository, and nothing is cited here until it has been read.

11. Progress so far

Phase 1 · 18 to 22 August · Done

- Proved the bypass is real, in a script anyone can run with no internet and no API key.
- Showed the filter catches 10 of 10 attacks, and the cache still answers 5 of them with the filter never running.
- Showed no threshold setting fixes it.
- Presented the result, with a deck generated from a recorded run, so no number on it can drift.

Phase 2, Week 1 · 23 to 29 August · Done

- Built the gateway: an OpenAI-compatible API behind nginx, in Docker.
- Added three ways to get a reply: a fixed test reply, a recorded reply, and a real model call.
- Added a daily spending cap, checked before every real call.
- Added one log line per request, with the settings that produced it and no question text.
- Started this paper and a weekly lab notebook.

Phase 2, Week 2 · 30 August to 5 September · Done

- Added the safety check. Every question gets a score, a personal data scan, an intent and a risk level.
- Attacks are refused with a clear error. Personal data is flagged, not blocked.
- The real attack model is wired in and switchable. The test suite uses a keyword stand-in, so it never downloads a model.

Phase 2, Week 3 · 6 to 12 September · Done

- Added the baseline cache: exact match first, then similarity search.
- Every stored answer carries its full safety result, ready for the Phase 3 defence.
- A cache hit is served without the filter, and the log records it.
- Added a way to reset the cache between experiment runs.
- 36 tests pass.

12. Still left

Phase 2 · by 26 September

- **Week 4, the main result of this phase.** Generate rewordings from public attack datasets, poison the cache, fire them at our own gateway, sweep the threshold, and read the bypass rate from the log.
- **A first real model reply through the gateway.** Two attempts on 9 September failed at the model provider. The gateway handled the failure correctly and logged it. A successful call is still to come.
- **Switch the gateway to the real components.** The real attack model, the full personal data scanner, and Qdrant and Redis, then measure their speed.
- **Week 5.** Complete the cost numbers in the log, generate the charts from it, redraw the architecture diagram, and record a backup demo.

Phase 3 · 27 September to 24 October

- Turn on the verdict check in the cache, and rerun the same attacks.
- Re-check high-risk cache hits before serving them.
- Draw the trade-off between safety and cache savings.
- Add the model picker, the output check and per-user spending limits.

Phase 4 · 25 October to 14 November

- Load testing, the full results table and the ablation study.
- Finish this paper: related work, results, conclusion.
- A final check that a fresh copy of the project rebuilds every number and chart with one command.